

Examen Práctico — Unidad 2

Arquitecturas Distribuidas: Microservicios, APIs RESTful y Contenerización

Aplicaciones Distribuidas (ISR-701)

Período académico 2026-2027

Asignatura	Aplicaciones Distribuidas (ISR-701)
Unidad	2 — Arquitecturas Distribuidas
Tipo de evaluación	Examen práctico de unidad — Individual
Modalidad	Laboratorio con defensa técnica oral
Duración	3 horas de laboratorio
Período académico	2026-2027
Código del instrumento	EV-PR-U2 (ajustable según planificación)
Docente	_____
Estudiante	_____
Fecha	_____
URL del repositorio	_____
Calificación (sobre 10)	_____

1. Competencia y resultado de aprendizaje evaluados

Este examen valora la capacidad del estudiante para **aplicar arquitecturas y patrones de diseño distribuido** —microservicios, APIs REST y contenerización con Docker— para construir un sistema funcional, desplegable y con separación clara de dominios. En términos concretos, se evalúa la habilidad para: (i) descomponer un dominio de negocio en microservicios independientes con su propia base de datos siguiendo el patrón *Database per Service* [1]; (ii) diseñar APIs RESTful conformes a las restricciones de Fielding [2], alcanzando al menos el Nivel 2 del modelo de madurez de Richardson en cuanto al uso disciplinado de verbos HTTP y códigos de estado; (iii) implementar comunicación inter-servicio síncrona con manejo básico de fallos; (iv) contenerizar cada microservicio con *multi-stage build* para minimizar la superficie de ataque y el tamaño de imagen [3]; y (v) orquestar el conjunto con Docker Compose, incluyendo red interna dedicada, *healthchecks* y *reverse proxy*, siguiendo patrones de despliegue documentados en la literatura [4].

2. Planteamiento del problema

Caso: MercadoQuevedo — Marketplace distribuido para comerciantes locales

MercadoQuevedo es una plataforma en línea, aún en fase de prototipo, para que pequeños comerciantes del cantón Quevedo publiquen productos y reciban pedidos. El emprendedor propietario ha reconocido que su implementación actual —un monolito Django— dificulta el crecimiento y solicita al equipo de Ingeniería de Software una **prueba de concepto** basada en microservicios que evidencie tres capacidades: separación de dominios, exposición RESTful bien diseñada y despliegue reproducible en contenedores.

El sistema, en su alcance mínimo, debe distinguir dos dominios de negocio:

- **Catálogo de productos:** altas, bajas, modificaciones y consultas de productos, con atributos mínimos (SKU, nombre, precio, *stock*).
- **Gestión de pedidos:** creación de pedidos que referencian uno o más productos, cálculo de subtotales, y consulta del estado del pedido.

La creación de un pedido requiere que el servicio de pedidos consulte al de catálogo para **verificar la existencia y disponibilidad** del producto. No se evalúa la calidad estética de la interfaz, la persistencia optimizada ni escenarios de alta concurrencia; se evalúa la **correcta aplicación de los estilos y patrones estudiados en la Unidad 2**.

Su tarea es construir un prototipo funcional del sistema y desplegarlo con Docker Compose, evidenciando los principios de la Unidad 2: separación en microservicios, diseño RESTful correcto, comunicación inter-servicio, contenerización con *multi-stage build* y orquestación con *reverse proxy*.

3. Requerimientos a implementar

Parte A — Arquitectura de microservicios y diseño en capas (peso orientativo 20 %)

- Implemente al menos **dos microservicios independientes**: `svc-catalogo` y `svc-pedidos`, cada uno con su propia base de datos (*Database per Service*) [1].
- Cada microservicio debe organizarse en capas `controller`, `service`, `repository` y `model` (o equivalente en el *stack* elegido), con separación clara de responsabilidades.
- Entregue un **diagrama de arquitectura** propio (C4 nivel 2 o UML de despliegue) que identifique: los microservicios, sus bases de datos, el *reverse proxy* y el flujo de peticiones del cliente.

Parte B — Diseño de la API REST (peso orientativo 25 %)

- Cada microservicio debe exponer una API REST que alcance como mínimo el **Nivel 2 del modelo de madurez de Richardson**: recursos identificados por URI, uso disciplinado de verbos HTTP (GET, POST, PUT/PATCH, DELETE) y códigos de estado apropiados.
- Las URIs deben seguir las convenciones REST: **sustantivos en plural**, versionadas (`/api/v1/...`), sin verbos en la ruta, con IDs en el *path* y no en *query strings*. Ejemplo esperado: `POST /api/v1/productos`.
- Los códigos de estado deben ser semánticamente correctos. Como mínimo: 201 Created en altas exitosas (con cabecera Location), 204 No Content en eliminaciones, 404 Not Found en recursos inexistentes, 409 Conflict en violaciones de unicidad (p. ej. SKU duplicado), 400 Bad Request en JSON malformado y 422 Unprocessable Entity en validaciones de dominio.
- Respete la **idempotencia** de PUT y DELETE, y la **no idempotencia** de POST.
- Publique la documentación OpenAPI 3.0 (Swagger UI o equivalente) para cada servicio.

Parte C — Comunicación entre servicios (peso orientativo 15 %)

- Al crear un pedido, `svc-pedidos` debe invocar por **HTTP a** `svc-catalogo` para verificar la existencia y disponibilidad del producto (comunicación síncrona basada en REST).
- Implemente al menos un mecanismo básico de **manejo de fallos** de esa llamada: *timeout* explícito y respuesta HTTP coherente al cliente si el catálogo no responde (por ejemplo, 503 Service Unavailable o 424 Failed Dependency).

- Si el producto no existe o no hay *stock* suficiente, el pedido debe rechazarse con el código HTTP y el cuerpo de error apropiados.

Parte D — Contenerización con Docker (peso orientativo 15 %)

- Cada microservicio debe contar con un `Dockerfile` **multi-stage**: una etapa de compilación (con las herramientas de *build*) y una etapa de *runtime* que solo contenga el JRE (o entorno de ejecución) y el artefacto empaquetado.
- La imagen final debe ejecutarse con **usuario no-root** y exponer únicamente el puerto necesario.
- Justifique la elección de la imagen base en el README (peso, mantenimiento, superficie de ataque).

Parte E — Orquestación con Docker Compose y *reverse proxy* (peso orientativo 15 %)

- Entregue un `docker-compose.yml` que levante: los dos microservicios, sus respectivas bases de datos y un **Nginx** (u otro *reverse proxy*) como puerta de entrada única.
- Defina una **red interna dedicada** (*bridge network*), **volúmenes persistentes** para los datos de las bases y **healthchecks** para todos los servicios.
- Los microservicios deben declarar `depends_on` sobre sus bases de datos con `condition: service_healthy`, de modo que solo arranquen cuando la base esté lista para aceptar conexiones.
- El *reverse proxy* debe enrutar externamente `/api/v1/productos/*` hacia `svc-catalogo` y `/api/v1/pedidos/` hacia `svc-pedidos`. Solo el puerto de Nginx debe estar expuesto al *host*.

4. Componente de análisis y defensa técnica

Al finalizar la implementación, el estudiante registrará sus respuestas en un archivo `ANALISIS.md` y las defenderá oralmente ante el docente. Las preguntas conectan el prototipo con la teoría de la unidad:

- ¿Cuál de las seis restricciones REST de Fielding [2] identifica con mayor claridad en su implementación? Argumente con al menos un extremo del código.
- ¿En qué nivel del modelo de madurez de Richardson se ubica su API? Justifique con un ejemplo concreto de recurso, verbo y código de estado que respalde ese nivel.
- Newman [3] sostiene que la elección entre monolito modular y microservicios debe hacerse en función de *coupling* y *cohesion*. Con base en su prototipo, ¿qué acoplamiento residual observa entre `svc-catalogo` y `svc-pedidos` y cómo lo mitigaría en producción?
- Blinowski et al. [5] reportan que, sobre una sola máquina, el monolito puede superar en rendimiento a la versión microservicios equivalente. Ante ese hallazgo, ¿qué justificación técnica sostendría la migración a microservicios en el caso de MercadoQuevedo?
- ¿Qué ganaría y qué perdería su prototipo si reemplazara la comunicación REST entre `svc-pedidos` y `svc-catalogo` por gRPC? Argumente en términos de rendimiento, tipado del contrato y consumidores externos.
- Si el proceso de `svc-catalogo` se detiene, ¿qué código HTTP recibe el cliente al crear un pedido y por qué? ¿Cómo introduciría un patrón *Circuit Breaker* [1] en esa integración?

5. Entregables y condiciones

- **Repositorio:** repositorio Git público (o con acceso al docente) con el código fuente de *ambos* microservicios, Dockerfiles, `docker-compose.yml`, configuración del *reverse proxy* y un `README.md` con instrucciones de compilación y despliegue completas.
- **Análisis:** archivo `ANALISIS.md` con las respuestas del componente de análisis.
- **Documentación de APIs:** especificación OpenAPI 3.0 exportada como YAML/JSON en `/docs/api/` del repositorio.
- **Demostración:** demostración en vivo: levantar el *stack* con `docker compose up`, ejecutar el flujo completo (crear producto, crear pedido, provocar un pedido que falle por producto inexistente, detener `svc-catalogo` y observar el comportamiento).
- **Condición:** el prototipo debe levantarse con un único comando (`docker compose up -build`) siguiendo el `README`. Un proyecto que no levanta el *stack* no puede acreditar los criterios funcionales.

6. Restricciones técnicas

- **Lenguaje y *stack* sugerido:** Java 21 + Spring Boot 3.x + PostgreSQL 16, o alternativa equivalente (Python 3.11+ con FastAPI, o Node.js 20 con NestJS) siempre que se respeten las capas y el diseño RESTful.
- **Contenerización obligatoria:** el sistema debe ejecutarse dentro de Docker Compose. No se aceptan ejecuciones “en local” o con IDE embebido durante la demostración.
- **IDE de referencia:** IntelliJ IDEA (Community o Ultimate). Docker Desktop debe estar operativo.
- El uso de asistentes de IA y de material de consulta se rige por la política indicada por el docente para esta evaluación. En la defensa, el estudiante debe poder explicar y justificar cada parte de su código sin leerlo textualmente.

7. Rúbrica de evaluación

Cada criterio se valora en cuatro niveles: Excelente (4), Bueno (3), Regular (2) y Deficiente (1). Un criterio no entregado, con plagio verificado o que no corresponde recibe 0. La calificación final se obtiene como la suma ponderada de los niveles alcanzados, según la fórmula:

$$\text{Nota (sobre 10)} = \left[\sum (\text{peso}_i \times \text{nivel}_i) \div 4 \right] \times 10$$

Donde los pesos se expresan como fracción (sumando 1,0) y el nivel es el valor 1 a 4 alcanzado en cada criterio. Si todos los criterios alcanzan el nivel Excelente, la nota es 10.

Rúbrica de evaluación — Examen Práctico Unidad 2

Criterio	Peso	Excelente (4)	Bueno (3)	Regular (2)	Deficiente (1)
C1. Arquitectura de microservicios y diseño en capas	20 %	Dos (o más) microservicios totalmente independientes, con <i>Database per Service</i> correctamente aplicado, capas bien separadas y diagrama de arquitectura propio, claro y coherente.	Microservicios separados con <i>Database per Service</i> ; alguna capa mezcla responsabilidades o el diagrama es genérico.	Los microservicios comparten base de datos o las capas están mezcladas; diagrama superficial.	Sistema monolítico con carpetas separadas; sin diagrama o incoherente.
C2. Diseño de la API REST (URIs, verbos y códigos de estado)	25 %	API en Nivel 2 (o superior) de Richardson: URIs consistentes (plural, versionadas, sin verbos), verbos HTTP con idempotencia respetada y códigos de estado correctos en <i>todos</i> los casos exigidos (201, 204, 400, 404, 409).	API en Nivel 2 con detalles menores: alguna URI inconsistente o un código de estado subóptimo.	API en Nivel 1: usa recursos pero mezcla verbos y códigos de estado (p. ej. 200 para todo).	API en Nivel 0 (un solo <i>endpoint</i> tipo RPC) o URIs con verbos y sin códigos de estado significativos.
C3. Comunicación entre servicios con manejo de fallos	15 %	<code>svc-pedidos</code> llama a <code>svc-catalogo</code> con <i>timeout</i> explícito; ante fallo del catálogo devuelve un código HTTP semánticamente correcto y consistente.	Comunicación funcional con <i>timeout</i> implícito o manejo de errores parcial.	Comunicación funciona en el caso feliz pero bloquea o falla sin control cuando el catálogo no responde.	No hay comunicación real entre servicios; los pedidos no validan producto o el flujo está codificado.
C4. Contenerización (Dockerfile <i>multi-stage</i>)	15 %	Dockerfile <i>multi-stage</i> para ambos servicios, imagen final ligera con solo JRE/JAR, usuario <code>no-root</code> y justificación de la imagen base en el README.	<i>Multi-stage</i> presente y funcional; falta el usuario <code>no-root</code> o la justificación de la imagen.	Dockerfile de una sola etapa; la imagen final incluye el JDK y herramientas de <i>build</i> .	Dockerfile ausente, roto o no reproducible.
C5. Orquestación con Docker Compose y <i>reverse proxy</i>	15 %	<code>docker-compose.yml</code> completo: red interna dedicada, volúmenes persistentes, <i>healthchecks</i> funcionales, <code>depends_on</code> con <code>service_healthy</code> y Nginx enrutando correctamente ambos servicios; solo el puerto del <i>reverse proxy</i> expuesto.	<i>Stack</i> operativo con detalles menores: falta un <i>healthcheck</i> , se expone un puerto extra o el enrutamiento tiene una regla incorrecta.	Los servicios levantan pero sin <i>healthchecks</i> , sin volúmenes persistentes o sin <i>reverse proxy</i> .	El <i>stack</i> no levanta o los servicios se comunican solo por IPs fijas y puertos publicados.

Criterio	Peso	Excelente (4)	Bueno (3)	Regular (2)	Deficiente (1)
C6. Calidad del código, documentación y repositorio	5 %	Repositorio con historial de <i>commits</i> coherente, código legible y organizado, README completo, especificación OpenAPI publicada y ejecutable con un solo comando.	Repositorio funcional con <i>README</i> claro; OpenAPI presente con detalles menores.	Ejecuta con dificultad o instrucciones incompletas; código desordenado.	No compila/ejecuta o no hay repositorio utilizable.
C7. Análisis y defensa técnica	5 %	Responde con precisión, cita conceptos correctamente (restricciones REST, nivel Richardson, <i>Circuit Breaker</i> , <i>trade-offs</i> REST/gRPC) y defiende su código sin leerlo, razonando ante preguntas inesperadas.	Análisis correcto en lo esencial con alguna imprecisión; defensa sólida.	Análisis superficial o parcialmente erróneo; defensa dubitativa.	No conecta la práctica con la teoría; no defiende su solución.

Condiciones que penalizan de forma no negociable

- Si el *stack* **no levanta** con `docker compose up -build`, C4 y C5 se califican como Deficiente (1) automáticamente, y C3 no puede superar Regular (2).
- Si las URIs contienen verbos en la ruta (p. ej. `POST /crearProducto`), C2 no puede superar Regular (2), independientemente de los demás criterios.
- Si los dos microservicios comparten la misma base de datos, C1 no puede superar Regular (2).
- Si el *reverse proxy* está ausente y los microservicios se exponen directamente al *host*, C5 no puede superar Regular (2).
- Plagio verificado en cualquier parte del entregable: calificación 0 en el examen.

Nota: “0 puntos” se asigna a un criterio no entregado, con plagio verificado o que no corresponde al criterio. La calificación final aplica la fórmula ponderada indicada en el encabezado de esta sección.

Referencias bibliográficas

Referencias

- [1] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018.
- [2] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [3] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2 ed., 2021.

- [4] I. Karabey Aksakalli, T. Çelik, A. B. Can, and B. Tekinerdoğan, "Deployment and communication patterns in microservice architectures: A systematic literature review," *Journal of Systems and Software*, vol. 180, p. 111014, 2021.
- [5] G. Blinowski, A. Ojdowska, and A. Przybylek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022.